Preste

Cómo lo implementamos, bounty por bounty, y el pitch

Monad Metropolis · Track 4 · Brenda Calizaya y Alan Meneces · 11 de octubre de 2026

Preste es una app de préstamos entre personas donde la huella es la cuenta. Corre sobre **Aval**, el protocolo de Alan para permisos y aprobaciones con passkey en Monad. Este documento explica, para cada bounty, qué piden, por qué las elegimos, qué librerías usamos, cómo lo hicimos paso a paso y dónde está en el código.

> **Bounties que quedan:** Best Mera-Powered UX · Mera: Una llave maestra, muchas llaves · Best Builds with Qwen 3.8 Max · Best Projects Using Alchemy · Best Community Team Project. **Quitadas:** Dynamic y Chainlink CRE (ver sección 8).

1. La arquitectura en una página

Capa	Qué hace	Librería o servicio	Dónde
App	Pantallas, rutas, API	**Next.js 16** (App Router), **React 19**, **Tailwind 4**	`app/src/app/`
Cuenta	Huella → cuenta de Monad, sesión de firma	**@category-labs/mera** 0.2, **@scure/bip32**, **@scure/bip39**, **viem** 2.57	`lib/aval/passkey.js`, `session.js`, `AvalProvider.jsx`
Protocolo	Permisos, aprobaciones con huella, recibos	**Aval**: `PasskeyRegistry`, `AgentPermit`, `ReputationReader`, `TestUSD` (SDK copiado en `src/vendor/aval-sdk`)	Monad testnet, chain 10143
Gas	El usuario no necesita MON	**Relayer de Aval** (`aval-relayer.vercel.app`) vía `createRelayerClient`	`api/relayer/[...ruta]`
Mercado	Billetera, ofertas, tratos, cuotas, calificaciones	Lógica pura + **@vercel/blob** (privado, control de concurrencia por ETag)	`lib/mercado/`, `api/mercado`
Autenticación	Cada petición firmada con la cuenta	`signMessage` de viem (EIP-191) + `recoverMessageAddress`	`lib/mercado/cliente.js`, `auth.js`
Memoria cifrada	La conversación del asistente, cifrada con la huella	**Web Crypto**: HKDF-SHA256 + AES-GCM 256, y **@vercel/blob**	`lib/memoria.js`, `api/memoria`
IA	Asistente con herramientas	**Qwen 3.8 Max** (Alibaba Cloud Model Studio, API compatible con OpenAI)	`api/agente/route.js`
Datos de cadena	RPC, movimientos, avisos	**Alchemy** (RPC y `alchemy_getAssetTransfers`)	`lib/aval/config.js`, `lib/alchemy.js`, `api/avisos`
QR	Calificación presencial	**qrcode** (generar) y **jsqr** (escanear)	`mi-codigo/`, `EscanerQR.jsx`
Hosting	App pública y almacén	**Vercel** (producción) y **Vercel Blob**	app-wheat-eta-38.vercel.app

Estado de lo implementado

Pieza	Estado
Cuenta con una huella, sin MON, sesión con bloqueo y reloj	Hecho
Mercado de préstamos con dinero de prueba (probado con 2 cuentas)	Hecho
Memoria cifrada con la huella	Hecho

Pieza	Estado
Aval: permisos y aprobaciones con huella	Hecho
Alchemy: RPC y lectura de movimientos	Hecho
Alchemy: webhook creado en el panel	Pendiente
Asistente con Qwen real (clave de Alibaba)	Pendiente
Artículo de Qwen, videos y textos del formulario	Pendiente
Prueba con huella real en dos dispositivos	Pendiente

> El asistente hoy corre en **modo simulado** (rotulado en pantalla "no es Qwen") hasta poner `DASHSCOPE_API_KEY` y `QWEN_MODEL`. No digas en los videos que corre Qwen hasta conectarlo.

2. Best Mera-Powered UX on Monad · $2,500

Qué piden

Una app donde **Mera es toda la capa de cuenta**: sin frase semilla, sin extensión, sin custodia. Se juzga el tiempo hasta la primera transacción (toques y segundos), el diseño de sesión, la prueba sin estado y, de bonus, combinar Mera con el resto del stack.

Por qué la elegimos

Es el corazón del producto: toda persona entra, firma y se recupera solo con su huella. Además, el gas lo paga un relayer, así que el usuario no nota que hay una blockchain.

Qué usamos

- **@category-labs/mera**: `createPasskeyWithPrfOutput`, `getPasskeyPrfOutput`, `createSecp256k1SigningSession`, `getEvmAddress` y `toViemAccount` (de `@category-labs/mera/viem`).
- **@scure/bip39** (`entropyToMnemonic`, `mnemonicToSeedSync`) y **@scure/bip32** (`HDKey`) para derivar la cuenta EVM de la salida PRF.
- **viem**: cliente de Monad (`createPublicClient`, `createWalletClient`, `fallback`) y firma de mensajes.
- **Aval SDK** y su **relayer**: registro de la clave P256 y gas patrocinado.

Cómo lo implementamos, paso a paso

1. **Una sola ceremonia.** Un cliente de WebAuthn propio (`lib/aval/passkey.js`, función `realBackend`) llama a `navigator.credentials.create` con ES256 y la extensión PRF, y **captura la clave pública P256** de la misma passkey. Mera la usa a través de `createPasskeyWithPrfOutput` (`lib/aval/session.js`, `createAccount`).
2. **La cuenta sale de la huella.** La salida PRF (32 bytes) pasa por BIP-39 y BIP-32 (`m/44'/60'/0'/0/0`) y da la clave de la cuenta (`openSession`). Con `createSecp256k1SigningSession` y `toViemAccount` se firma **sin ventanas**.
3. **Sin MON.** El relayer entrega tUSD de prueba y registra la clave P256 en `PasskeyRegistry` (`preparar` en `AvalProvider.jsx`: `relayer.faucet` y `passkeys.signRegister` → `relayer.register`).
4. **Medición.** El proveedor cuenta los toques, los segundos hasta la cuenta lista y la **primera transacción confirmada** (`medir`, `primeraTx`, `toques`). "Cuenta lista" lo muestra con un enlace al explorador.
5. **Sesión con alcance.** Sin huella: ver saldos, crear permisos, publicar, hablar con el asistente. Con huella siempre: aprobar un pago grande (`aprobar`) y volver a entrar. La sesión vive solo en memoria y se bloquea a los 10 minutos (`tocar`, `bloquear`, variable `NEXT_PUBLIC_SESSION_MINUTES`).
6. **Expiración clara.** `ChipSesion.jsx` muestra una cuenta regresiva en el encabezado, avisa un minuto antes y permite mantener o bloquear. `GuardiaSesion.jsx` muestra "Continuar con huella".
7. **Prueba sin estado.** `signIn` (`getPasskeyPrfOutput`) reconstruye la misma cuenta; todo lo demás se lee de la cadena (`cargar`) y del servidor. No se guarda nada en `localStorage` (solo el modo "sin passkey", rotulado). El botón "Probar como dispositivo nuevo" (`olvidar`) lo demuestra.
8. **Peticiones firmadas.** Cada acción del mercado se firma con `account.signMessage` y el servidor recupera la dirección (`recoverMessageAddress`): no hay contraseñas ni tokens (`lib/mercado/cliente.js` y `auth.js`).

Resultado medido (con clave de prueba)

Cuenta lista en ~5 s, **1 toque**, 1 verificación de huella, primera transacción confirmada en ~3 s. Con huella real hay que volver a medir.

Texto para el formulario

"Preste usa Mera como toda la capa de cuenta. Una sola ceremonia de passkey (`createPasskeyWithPrfOutput`) entrega la salida PRF, de la que se deriva la cuenta EVM y se abre una sesión de firma en memoria. En esa misma ceremonia se captura la clave P256 de la passkey y se registra en la cadena, y un relayer patrocina el gas: el usuario nunca necesita MON. La sesión firma sin avisos las acciones cotidianas y se bloquea sola por inactividad, con reloj visible; aprobar un pago grande siempre pide la huella. Al borrar el almacenamiento o abrir otro dispositivo, la misma huella reconstruye cuenta, permisos, recibos y memoria desde la cadena."

3. Mera: Una llave maestra, muchas llaves · $2,500

Qué piden

El uso más creativo de la salida PRF de Mera para algo que **no sea firmar con una cuenta**: llaves independientes por espacio de nombres, cifrado y derivación bien usados, nada sensible guardado, y que **en otro dispositivo se reproduzcan las mismas llaves** y se descifre el mismo estado.

Por qué la elegimos

Encaja con el asistente: su **memoria** (lo que conversaste) es privada y debe sobrevivir en cualquier dispositivo sin que nadie más la lea, ni siquiera nosotros. Es la idea 02 que sugiere el bounty.

Qué usamos

- **Web Crypto API** del navegador: `crypto.subtle.importKey` + `deriveBits` con **HKDF-SHA256**, y **AES-GCM de 256 bits** (`encrypt` y `decrypt`).
- **@category-labs/mera** para obtener la salida PRF (la misma ceremonia de la cuenta).
- **@vercel/blob** (almacén privado `preste-memoria`) para guardar solo el texto cifrado.

Cómo lo implementamos, paso a paso

1. **Derivar dos espacios de nombres** de la misma passkey, con una etiqueta HKDF distinta cada uno (`lib/memoria.js`, `derivarEspacios`): `preste/memoria/clave/v1` (clave AES) y `preste/memoria/id/v1` (identificador anónimo). Conocer una no revela la otra ni la clave de la cuenta.
2. **Cifrar en el navegador** (`cifrar`): AES-GCM con IV aleatorio, y el identificador como dato autenticado, así un texto cifrado de otra memoria no se puede pasar por esta. La clave se importa **no extraíble** y se borra la copia cruda.
3. **Guardar bajo un identificador que no es la dirección** (`api/memoria`, `PUT`): el servidor solo recibe `{id, v, iv, ct}`. No puede leer la conversación ni saber de quién es.
4. **Recuperar al entrar** (`AvalProvider.jsx`: `leerMemoria`, `guardarMemoria`, `borrarMemoria`; `Asistente.jsx`): en otro dispositivo o navegador, la misma huella deriva las mismas claves y descifra el mismo estado.
5. **Hacerlo visible.** La tarjeta "Memoria cifrada con tu huella" y su desplegable "Qué ve el servidor" muestran el identificador, el tamaño cifrado y el almacén. La pantalla Actividad lista las llaves derivadas.

Verificado

Una conversación en una pestaña y, en otra **sin nada en memoria**, la misma clave recuperó los mensajes desde Blob. El servidor devolvió solo `{v, iv, ct}`, sin el texto del chat.

> **Aclaración honesta:** los espacios de nombres son etiquetas de HKDF sobre una sola salida PRF, no varias sales distintas de PRF. Cumple "claves independientes por espacio de nombres" sin pedir una huella extra, pero dilo así en el texto. Además, el mensaje pasa en claro por el servidor hacia Qwen mientras se responde; no se guarda ahí.

Texto para el formulario

"De una sola passkey derivamos, con HKDF-SHA256 y etiquetas distintas, una clave AES-GCM que cifra la memoria del asistente y un identificador anónimo donde se guarda, separado de la cuenta. El navegador cifra; el servidor (Vercel Blob) solo guarda texto cifrado y no puede leerlo ni ligarlo al usuario. Con la misma huella en otro dispositivo, la memoria se descifra igual, sin guardar ningún secreto."

4. Best Builds with Qwen 3.8 Max · $5,000 en créditos

Qué piden

Uso agéntico real de Qwen: planificar, usar herramientas y ejecutar varios pasos; un producto funcionando en Monad; **integración profunda** y un **artículo publicado** sobre cómo se usó.

Por qué la elegimos

El asistente no es un chat: busca ofertas con un presupuesto y una moneda, resume la cuenta y puede mostrar interés, llamando a herramientas que leen y cambian el estado real. Es la forma natural de una persona de decir "necesito 200 Bs" sin recorrer el Tablón.

Qué usamos

- **Qwen 3.8 Max** en **Alibaba Cloud Model Studio**, por su API compatible con OpenAI (`https://dashscope-intl.aliyuncs.com/compatible-mode/v1/chat/completions`), con llamadas a funciones (`tools`).
- **Variables de entorno de servidor**: `DASHSCOPE_API_KEY`, `DASHSCOPE_BASE_URL`, `QWEN_MODEL`. La clave nunca llega al navegador.
- **viem** (`recoverMessageAddress`) para saber quién pregunta; **Aval SDK** para las herramientas de pagos opcionales.

Cómo lo implementamos, paso a paso

1. **Identidad.** Cada petición al asistente va firmada con la cuenta (`peticionFirmada`, `verificarPeticion`). El servidor sabe quién pregunta sin contraseñas.
2. **Herramientas del mercado** (siempre disponibles, `api/agente/route.js`): `buscar_ofertas` (presupuesto, moneda BOB/USD, tipo, plazo y tasa máximos), `mi_resumen` (saldo, tratos que requieren atención) y `me_interesa` (solo si el usuario lo pide). **No requieren ningún permiso de gasto.**
3. **Bucle agéntico.** `conQwen` manda el historial y las herramientas a Qwen; si responde con `tool_calls`, ejecuta cada una en el servidor, devuelve el resultado como mensaje `tool` y repite hasta 6 pasos. La respuesta final incluye los pasos, que la interfaz muestra ("Lo que hizo") y, para las ofertas, como tarjetas con enlace.
4. **Pagos autorizados, aparte (Aval).** Solo si el usuario activa su permiso se suman `consultar_permiso`, `pagar` y `pedir_aprobacion`. Los pagos pequeños se ejecutan; los grandes quedan pendientes hasta que el usuario los apruebe con su huella (P256 verificada en la cadena).
5. **Memoria.** La conversación se cifra y se recupera (sección 3).
6. **Conectarlo.** Poner las tres variables en Vercel y quitar `AGENTE_SIMULADO`. Mientras tanto corre un planificador de reglas con las mismas herramientas, rotulado "no es Qwen".

Lo que falta para esta bounty

- La clave de Alibaba Cloud y el identificador del modelo.
- Probar con Qwen real y grabar la demo.
- **El artículo publicado** (requisito). Guion abajo, en la sección 9.

5. Best Projects Using Alchemy · $1,000 en créditos

Qué piden

Un proyecto desplegado en Monad que integre **de forma significativa** al menos un servicio de Alchemy.

Qué usamos

- **Alchemy RPC para Monad Testnet** (`https://monad-testnet.g.alchemy.com/v2/<KEY>`) como primer transporte de viem (`fallback`) en `lib/aval/config.js`, con RPC públicos de respaldo.
- **`alchemy_getAssetTransfers`** (API de transferencias) para leer los movimientos de tUSD de una cuenta (`lib/alchemy.js`, `actividadToken`).
- **Webhooks de Alchemy**: receptor con verificación de firma HMAC-SHA256 y suscripción de direcciones (`api/avisos`, `api/avisos/suscribir`).

Cómo lo implementamos, paso a paso

1. Crear la app "preste" en el panel de Alchemy con la red Monad Testnet y guardar la URL como `NEXT_PUBLIC_RPC_URL` (también en las variables de Vercel).
2. `config.js` arma `fallback([http(alchemy), http(público…)])`: toda lectura y envío de la app pasa primero por Alchemy.
3. En **Actividad**, `actividadToken` pide `alchemy_getAssetTransfers` con `category: ["erc20"]` y el contrato de tUSD, en ambos sentidos, y muestra "Salió / Recibiste" con enlace al explorador. Sin Alchemy habría que escanear bloques uno por uno.
4. La ruta `/api/avisos` recibe el webhook, calcula `HMAC-SHA256` sobre el cuerpo con la clave de firma (`ALCHEMY_WEBHOOK_SIGNING_KEY`) y rechaza lo que no coincide; probamos 200 con firma válida y 401 con firma falsa.

Lo que falta

- Crear el webhook de tipo Address Activity en el panel apuntando a `https://app-wheat-eta-38.vercel.app/api/avisos`, y copiar su clave de firma, el ID y el token a Vercel.
- El texto de entrega y, si quieres, el video de 2 minutos mostrando las llamadas en el panel de Alchemy (Request Logs).

6. Best Community Team Project · $5,000

No lleva código. Es un campo del **perfil del equipo en el portal de Monad Metropolis**: hay que indicar el grupo o campus al que pertenecen y elegir la comunidad en la lista de las registradas. Si la tuya no aparece, no se puede postular; pregunta en el Discord de Metropolis. En el formulario de entrega piden "¿Qué comunidad representa tu equipo?".

7. Track 4 · Trust, Identity & AI Infrastructure

El Track 4 pide **un protocolo o capa de infraestructura** sobre la que se construyen otras apps, no un producto aislado. Por eso presentamos dos partes:

Parte	Quién	Qué es
Aval (protocolo)	Alan	Contratos en Monad testnet (`PasskeyRegistry`, `AgentPermit`, `ReputationReader`, `TestUSD`), SDK `@aval/sdk` y relayer que paga el gas. Verifica la huella en la cadena con el **precompile P256 (`0x0100`)** y usa los registros **ERC-8004** para la reputación de agentes.
Preste (la app)	Brenda	Cuenta con Mera, mercado de préstamos con reputación, asistente con Qwen y los pagos autorizados con Aval. Usa Aval **solo por su SDK**, sin tocar los contratos: otra app integra el protocolo con pocas líneas.

Qué es de la cadena y qué no (dilo así)

- **En Monad testnet:** la cuenta y la clave P256 registrada, los permisos y las aprobaciones con huella, los recibos y la reputación de agentes (Aval).
- **En Vercel Blob:** los saldos en Bs de prueba, las ofertas, los tratos y las calificaciones del mercado, con peticiones firmadas por la cuenta. Es dinero de prueba, sin custodia real.
- La reputación de personas sale de tratos pagados a tiempo y de calificaciones con **QR presencial que cambia cada 30 s** (firmado con la cuenta, de un solo uso).

8. Bounties que quitamos y por qué

Bounty	Motivo
Dynamic ($5,000)	Pide integrar el SDK de Dynamic para autenticación o billeteras. Choca con "Mera es toda la capa de cuenta" de las bounties de Mera, y en el tiempo que queda no se puede hacer con profundidad (ellos premian profundidad sobre amplitud).
Chainlink CRE ($3,000)	Es un flujo aparte con su propia CLI y entorno. Con las entregas pendientes, el costo de hacerlo bien supera el beneficio. Preferimos terminar bien las cinco que quedan.

En el panel del portal hay que pulsar **Eliminar** en esas dos para que no cuenten como pendientes de entrega.

9. Pitch

Versión corta en español (45 segundos)

"En Bolivia se presta de palabra y la palabra no deja rastro. **Preste** convierte esos préstamos en historial verificable. Entras solo con tu huella: **Mera** deriva tu cuenta de la passkey y un relayer paga el gas, así que no necesitas MON. Publicas o pides un préstamo, la otra persona se interesa, se aceptan las condiciones, se desembolsa y se paga por cuotas. Al terminar, se califican **frente a frente con un código QR que cambia cada 30 segundos**, firmado por su cuenta. Un asistente con **Qwen** busca ofertas según tu presupuesto en bolivianos o dólares, y su memoria está **cifrada con claves derivadas de tu huella**: el servidor no puede leerla y en otro dispositivo vuelve igual. Todo corre sobre **Aval**, un protocolo en Monad que verifica la huella en la cadena con el precompile P256. Tu palabra ahora tiene historial."

English pitch (60 seconds)

"In Bolivia, millions of loans happen face to face, on trust alone, and trust leaves no trace. **Preste** turns them into a verifiable history. You sign in with a fingerprint only: **Mera** derives your account from the passkey, and a relayer sponsors gas, so you never need MON. Post or request a loan, get matched, both sides accept, funds are disbursed and repaid in installments. Afterwards you rate each other **in person with a QR code that rotates every 30 seconds**, signed by each account. A **Qwen 3.8 Max** agent plans and calls tools to find offers within your budget, in bolivianos or dollars, and its memory is **encrypted with keys derived from your passkey** through **HKDF and AES-GCM**: our server only stores ciphertext, and any new device with the same passkey decrypts it. Under the hood is **Aval**, a protocol on Monad that verifies the fingerprint onchain with the **P256 precompile**, plus **Alchemy** for RPC and transfer history. Your word now has a history."

Guion del video de 2 minutos (con los nombres de lo que se ve)

Tiempo	Qué se ve	Qué decir (y nombrar)
0:00–0:15	Inicio, un toque en "Crear cuenta con mi huella"	Una ceremonia de passkey con **Mera**; sin frase semilla, sin extensión, sin MON.
0:15–0:30	"Cuenta lista": segundos, toques, primera transacción	El relayer de **Aval** registró la clave **P256** y pagó el gas. Primera transacción confirmada en Monad.
0:30–1:00	Billetera, Tablón, trato: publicar, interés, aceptar, desembolsar, abonar	Cada acción va firmada con la cuenta (**viem**, `signMessage`) sin ventanas de huella. El estado vive en **Vercel Blob**.
1:00–1:20	Mi código y calificar	QR que cambia cada 30 s (**qrcode** y **jsqr**), firmado, de un solo uso: prueba de que están frente a frente.
1:20–1:45	Asistente: "ofertas con 30 dólares"	Agente de **Qwen 3.8 Max** con herramientas (`buscar_ofertas`, `mi_resumen`). No pide permisos. Su memoria se cifra con **HKDF y AES-GCM** derivados de la huella.
1:45–2:00	Actividad y cierre	Movimientos con **Alchemy** (`alchemy_getAssetTransfers`), sesión con reloj, prueba sin estado. "Tu palabra ahora tiene historial."

Guion del artículo de Qwen (requisito de la bounty)

- **Título sugerido:** "Un asistente de préstamos con Qwen 3.8 Max que no puede leer tus conversaciones".
- **Estructura:** el problema (préstamos informales en Bolivia) → por qué un agente y no un buscador → las herramientas (`buscar_ofertas`, `mi_resumen`, `me_interesa`) con su esquema JSON → el bucle de planificación y los 6 pasos máximos → qué valor aportó Qwen (entender "30 dólares" y "hasta 200 Bs", comparar, explicar) → separar consultar de pagar (permisos de Aval) → memoria cifrada con la huella → límites y lo que aprendimos.

- **Incluir:** capturas del paso a paso y un fragmento del código de `conQwen`. Publicarlo en dev.to, Medium o el blog del equipo y poner el enlace en el formulario.

10. Lista final antes de entregar

Tarea	Quién	Estado
Probar con huella real en celular y Chrome; y entre dos dispositivos	Brenda	Pendiente
Poner `DASHSCOPE_API_KEY` y `QWEN_MODEL` en Vercel y quitar `AGENTE_SIMULADO`; probar	Brenda	Pendiente
Crear el webhook de Alchemy y poner sus datos en Vercel	Brenda	Pendiente
Escribir y publicar el artículo de Qwen	Brenda	Pendiente
Eliminar Dynamic y CRE del panel; marcar la comunidad (Community Team)	Brenda	Pendiente
Commit y push del código; repo público con acceso a `metropolis@hackathon.monad.xyz`	Los dos	Pendiente
SDK de Aval publicado en npm y contratos verificados	Alan	Pendiente
Vaciar los datos de prueba de producción antes de grabar	Brenda	Pendiente
Grabar demo (3 min), pitch (2 min) y video de Mera (2 min)	Los dos	Pendiente
Textos de cada bounty en el formulario (están en las secciones 2 a 5)	Los dos	Pendiente
Confirmar que el relayer tiene fondos (`/health` en `"ok": true`)	Alan	Vigilar
Enviar antes de las 18:00 del 13 de octubre	Los dos	Pendiente